# Circuitos Digitais (Flip-flops)

## Conversão entre tipos

Prof. Gortan

# Introdução

Em circuitos digitais foram estudados vários tipos de flip-flops: D, RS, T, JK.

Em geral, os circuitos de lógica configurável dispõem de flip-flops tipo D em seus elementos lógicos ou suas macrocélulas.

Como converter de um tipo para outro?

# Conversão D para JK

Montamos a tabela da verdade de conversão de J e K para D: por exemplo:

- Se J = K = 1 ocorrerá um toggle, logo D = ~Q.
- Se J = 1 e K = 0 a saída será zerada, logo D = 0
- Etc.. – dando origem à tabela da verdade abaixo:

| J | K | D |
|---|---|---|
| 0 | 0 | Q |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | ~Q |

- A partir dessa tabela montamos a tabela de dependências de D com J, K e Q (próximo slide)

# Conversão D para JK

Montamos a tabela da verdade de D em função de J e K e Q:

| J | K | Q | D |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

# Conversão D para JK

Com o auxílio do Logisim montamos o mapa de Karnaugh e obtemos a expressão simplificada:

| J | K | Q | D |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

Saída: D

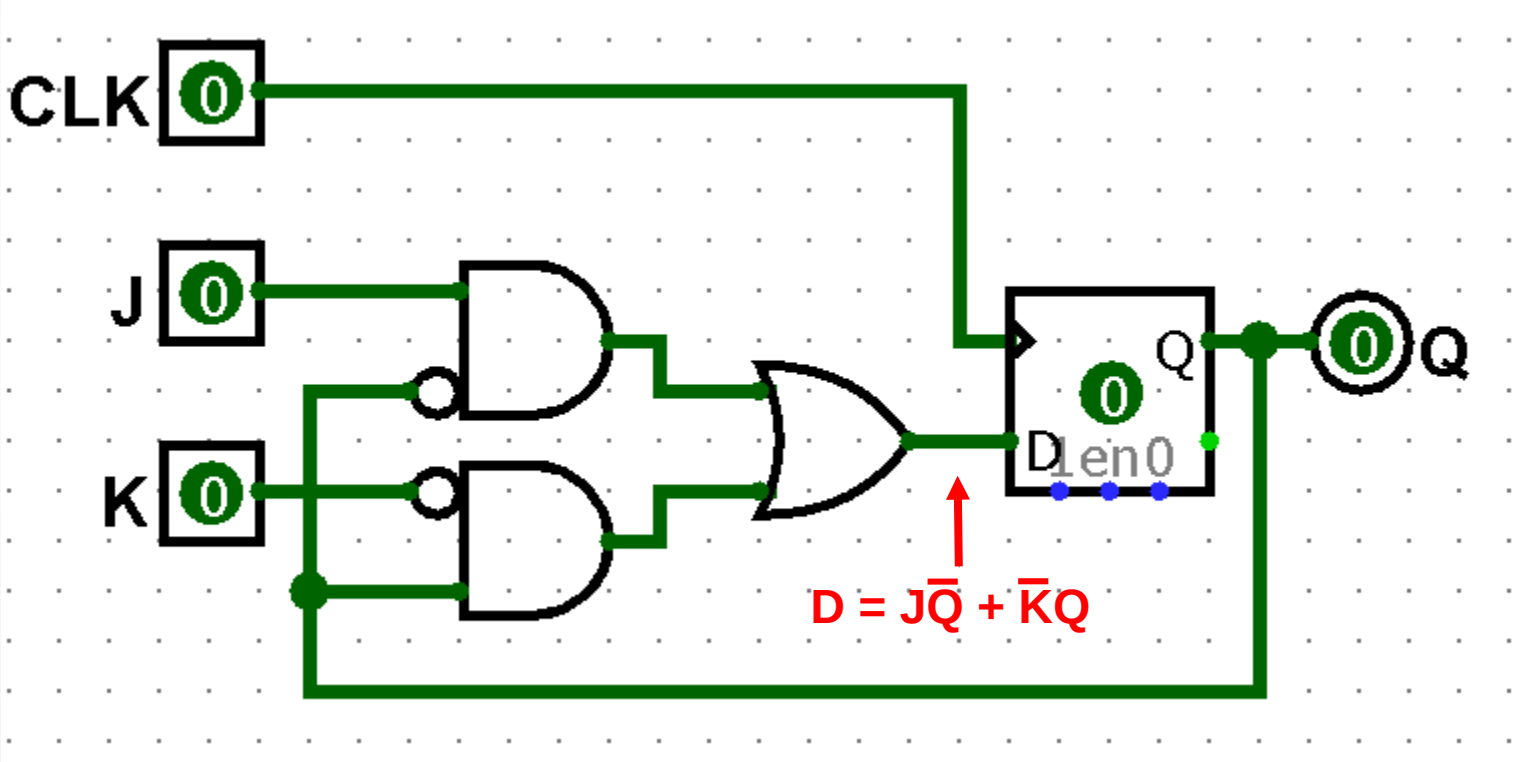
Formato: Soma de produtos

| J \ K, Q | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 |

$$\overline{K}Q + J\overline{Q}$$

# Conversão D para JK

Circuito implementado com portas:



# Conversão D para JK

Circuito implementado em VHDL (versão descritiva):

```
entity ff_jk is
  port(
    J, K, CLRN, PRN, CLK  : in  std_logic;  -- CLK, J e K e clear e preset
  ---------------------------------------------------------------------------
    Q, QN : out std_logic                   -- Saídas Q e Q invertido
    );
end ff_jk;
---------------------------------------------------------------------------
architecture ff_jk_arch of ff_jk is

signal qstate: std_logic;
begin
    process(clk, prn, clrn)                 -- sensitivity list
    begin
        if clrn = '0' then qstate <= '0';
        elsif prn = '0' then qstate <= '1';
        elsif clk = '0' and clk' event then -- detecção de borda negativa
            if j = '1' and k = '1' then qstate <= not qstate;
            elsif j = '1' and k = '0' then qstate <= '1';
            elsif j = '0' and k = '1' then qstate <= '0';
            end if;
        end if;
    end process;
    q <= qstate;
    qn <= not qstate;
end ff_jk_arch;
```

# Conversão D para JK

Circuito implementado em VHDL (expressão lógica):

```
entity ff_jk is
  port(
    J, K, CLRN, PRN, CLK  : in  std_logic;  -- CLK, J e K e clear e preset
  ---------------------------------------------------------------------------
    Q, QN : out std_logic                   -- Saídas Q e Q invertido
    );
end ff_jk;
---------------------------------------------------------------------------
architecture ff_jk_arch of ff_jk is

signal qstate: std_logic;
begin
    process(clk, prn, clrn)                 -- sensitivity list
    begin
        if clrn = '0' then qstate <= '0';
        elsif prn = '0' then qstate <= '1';
        elsif clk = '0' and clk' event then -- detecção de borda negativa
            qstate <= (not k and qstate) or (not qstate and j);
        end if;
    end process;
    q <= qstate;
    qn <= not qstate;
end ff_jk_arch;
```

$D = J\overline{Q} + \overline{K}Q$

# Conversão D para T

Montamos a tabela da verdade de D em função de T e Q:

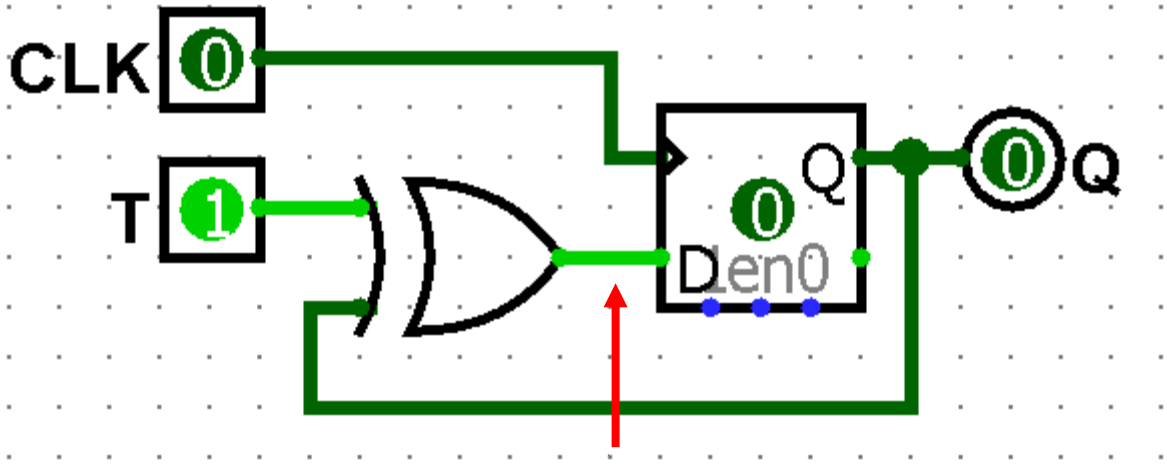
| T | Q | D |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Nesse caso a tabela é muito simples, e permite identificar imediatamente que D = T xor Q

$$D = T\overline{Q} + \overline{T}Q$$

# Conversão D para T

Circuito




$D = T\overline{Q} + \overline{T}Q$

# Conversão D para T

Circuito implementado em VHDL (versão descritiva):

```
entity ff_t is
  port(
    T, CLRN, PRN, CLK    : in  std_logic;        -- CLK, T e clear e preset
  ---------------------------------------------------------------------------
    Q, QN                : out std_logic         -- Saídas Q e Q invertido
    );
end ff_t;
---------------------------------------------------------------------------
architecture ff_t_arch of ff_t is

signal qstate: std_logic;
begin
    process(clk, prn, clrn)
    begin
        if clrn = '0' then qstate <= '0';
        elsif prn = '0' then qstate <= '1';
        elsif clk = '1' and clk' event then
            if T = '1' then qstate <= not qstate;
            end if;
        end if;
    end process;
    q <= qstate;
    qn <= not qstate;
end ff_t_arch;
```

# Conversão D para T

Circuito implementado em VHDL (expressão lógica):

```
entity ff_t is
  port(
    T, CLRN, PRN, CLK    : in  std_logic;        -- CLK, T e clear e preset
  ---------------------------------------------------------------------------
    Q, QN                : out std_logic         -- Saídas Q e Q invertido
    );
end ff_t;
---------------------------------------------------------------------------
architecture ff_t_arch of ff_t is

signal qstate: std_logic;
begin
    process(clk, prn, clrn)
    begin
        if clrn = '0' then qstate <= '0';
        elsif prn = '0' then qstate <= '1';
        elsif clk = '1' and clk' event then qstate <= T xor qstate;
        end if;
    end process;
    q <= qstate;
    qn <= not qstate;
end ff_t_arch;
```

$D = T\overline{Q} + \overline{T}Q$